

B1: Runology

CS1101S: Programming Methodology

Boyd Anderson

August 19th, 2023

- 1 First Mission
- 2 Recap
- 3 Source in Pictures

Announcements

- This Monday/Tuesday 2-h **S**tudio session S2 on Elements of Programming
- Don't worry if you have not been assigned a session. We will have an online option on Tuesday at 6pm-8pm.
- The game is open and the first chapter is out!
- Don't forget about our ED forums:
<https://edstem.org/us/courses/42391/discussion/>

- 1 First Mission
- 2 Recap
- 3 Source in Pictures

First Mission: Rune Trials

- ...will be released at 10:45 **today**.
- Missions are submitted individually and carry significant XP.
- Path P1B will be released at 10:45 today.
- You *can* discuss paths, missions, and quests with your friends and avengers.
- You *must* do the actual programming yourself. Do *not* copy anyone's programs, from other students or from previous batches or using AI.
- Submitting other people's work as your own is a serious offence and will be severely punished by NUS.

- 1 First Mission
- 2 **Recap**
 - Evaluating combinations, 1.1.3
- 3 Source in Pictures

Recap: Elements of Programming (1.1)

- Primitives: things like `-42` or `false`
- Combination: things like `-42 * 7`
- Name abstraction: things like `const size = 2;`
- Conditional expressions:
 `predicate ? consequent : alternate`
- Functional abstraction: more on this today

A function can be a “black box”

Example

`math_sqrt`

- Input: any number
- Output: a number whose square is the input number

Function application

`math_sqrt(15);`

Means of Abstraction: Compound functions

Example

```
function square(x) {  
    return x * x;  
}
```

What does this statement mean?

Like constant declarations, this *function declaration* declares a name, here square. The value associated with the name is a function that takes an argument *x* and produces (returns) the result of multiplying *x* by itself.

Evaluation of expressions

1 + 2 * 3 + 4

Replace “eligible” subexpression with result

(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4

11

Story: <https://sourceacademy.nus.edu.sg/stories/view/1>

Evaluation of constant declarations

```
const a = 3 * 5;  
a + 5;
```

Evaluate what is after the =

```
const size = 3;  
2 * size;
```

Then replace that name in rest of the program with that value.

```
2 * 3;  
  
6;
```

Story: <https://sourceacademy.nus.edu.sg/stories/view/2>

Evaluation of conditional expressions

```
3 < 4 ? 1 : 2;
```

Evaluate the predicate first.

```
true ? 1 : 2;
```

If the predicate evaluates to true, evaluate the consequent

```
1;
```

If the predicate evaluates to false, evaluate the alternate

```
3 > 4 ? 1 : 2;
```

```
false ? 1 : 2;
```

```
2;
```

- 1 First Mission
- 2 Recap
- 3 Source in Pictures**

Primitives

Some **simple runes** such as

- `rcross`
- `sail`
- `corner`
- `nova`
- `heart`
- `show` is a primitive operation that displays a rune in the REPL.

Primitive Operations

Transformations

Functions that take a rune and produce a new rune from it

Example transformation

Turn a given rune a quarter turn right: `quarter_turn_right`

Example in Source

```
quarter_turn_right(quarter_turn_right(sail))
```

Abstraction: Functions and naming

```
function turn_upside_down(rune) {  
    return quarter_turn_right(  
        quarter_turn_right(rune));  
}  
  
// example use:  
show(turn_upside_down(heart));
```


Abstraction: Functions and naming (continued)

```
function quarter_turn_left(rune) {  
    return quarter_turn_right(  
        turn_upside_down(rune));  
}  
  
// example use:  
show(quarter_turn_left(heart));
```

Combination operation: **stacking**

```
stack(rcross, sail);
```

Your turn: Combination abstraction

```
function beside(rune1, rune2) {  
    ???  
}
```

```
// example use:  
show(beside(rcross, sail));  
// should show rcross on the left  
//           and sail on the right
```

Solution: Combination abstraction

```
function beside(rune1, rune2) {  
    return quarter_turn_left(  
        stack(quarter_turn_right(rune1),  
            quarter_turn_right(rune2)));  
}
```

Combination: stacking something n times

```
stackn(5, heart);
```

Lecture L2

We will define `stackn` in `Source`, using rune primitives.

Naming and more combination: rectangular quilting

```
const my_quilt =  
    stackn(5,  
        quarter_turn_right(  
            stackn(7, quarter_turn_left(heart))));
```

Abstraction: rectangular quilting

```
function quilt(n, m, rune) {  
    return stackn(n,  
        quarter_turn_right(  
            stackn(m,  
                quarter_turn_left(  
                    rune))));  
}
```

Combination: a more complex rune

```
stack(beside(quarter_turn_right(rcross),  
             turn_upside_down(rcross)),  
      beside(rcross,  
             quarter_turn_left(rcross)));
```


Naming interesting things

```
const my_cross =  
    stack(beside(quarter_turn_right(rcross),  
                 turn_upside_down(rcross)),  
          beside(rcross,  
                 quarter_turn_left(rcross)));
```

Abstraction: making a cross from any rune

```
function make_cross(rune) {  
    return stack(beside(  
        quarter_turn_right(rune),  
        turn_upside_down(rune)),  
        beside(  
            rune,  
            quarter_turn_left(rune)))  
}
```

make_cross is a transformation.

Combination: repeating a transformation

```
make_cross(make_cross(nova));
```

Abstraction: repeating a transformation n times

```
repeat_pattern(5, make_cross, rcross);
```

Lecture L2

Like `stackn`, we will define this abstraction in Source next Wednesday!

Recap

- We started with primitive constants.
- No idea how they are rendered!
Example: `heart`
- We introduced primitive combinations.
- No idea how the primitive combinations work!
Example: `quarter_turn_right`
- Yet we can use primitives and combinations to generate complex runes.
- **Abstractions** allow us to master complexity:
Naming and **functional abstraction**
- Story: <https://sourceacademy.nus.edu.sg/stories/view/3>
(stories are an experimental feature)